FSP Talent

Платформа подбора ИТ-специалистов с подтверждённым грейдом

ЛЦТ-2026, спецтрек ФСП · МГТУ ГА

Сопроводительная документация

0. Краткое описание подхода

Задача. Платформа подбора ИТ-специалистов, где работодатель сам выходит на кандидата в категоризированной базе и приглашает его с указанием зарплаты.

Механика подбора — «заявка на команду». Работодатель описывает потребность: специализацию, грейд, навыки с весами, зарплату от–до с пометкой «гросс» или «нетто», формат работы. Система показывает подходящие категории («специализация + грейд») с числом подтверждённых кандидатов и ранжированную подборку. У каждого места в подборке есть объяснение «почему он здесь»: какие навыки совпали, какой грейд подтверждён тестом и на каком процентиле категории, какие результаты ФСП учтены. Работодатель отправляет адресное приглашение с зарплатой; контакты кандидата открываются только после того, как кандидат принял приглашение или сам откликнулся на вакансию.

Механика тестирования. Грейд кандидата подтверждается только экспресс-тестом по категории (10 заданий, 5 минут). Задания — параметрические шаблоны: каждый шаблон по сиду кандидата генерирует свои числа, имена, фрагменты кода и порядок вариантов, правильный ответ вычисляет код. Сложность задаёт шаблон, а не случайность, поэтому разные варианты сопоставимы. Уровень кандидата оценивается по модели Раша, задания подбираются адаптивно. Слитые ответы бесполезны: у другого кандидата другие варианты. Это проверено симуляцией (раздел 4).

Почему так. Резюме можно приукрасить, а одинаковый для всех тест быстро утекает в сеть. Персональные варианты с калиброванной сложностью дают сопоставимую оценку и устойчивы к утечке. Результаты ФСП учитываются как дополнительный сигнал только внутри категории, поэтому кандидат без истории ФСП проходит весь путь наравне с другими.

1. Функциональная и компонентная архитектура

Компоненты

- **Браузер** — веб-приложение (Vite + React + TypeScript) с кабинетами кандидата и работодателя.
- **nginx** (контейнер `frontend`) — отдаёт статику и проксирует `/api` на бэкенд.
- **FastAPI** (контейнер `backend`, порт 8000) — API бизнес-логики, OpenAPI на `/docs`.
- **PostgreSQL 16** (контейнер `db`) — хранение; схема ведётся миграциями alembic, при старте бэкенда выполняются `alembic upgrade head` и загрузка демо-данных.

Всё поднимается одним `docker-compose.yml`. GPU и LLM в обязательном пути не используются.

Модули бэкенда

Три модуля без взаимных импортов — обмен только словарями и pydantic-схемами, роутеры FastAPI тонкие:

Модуль	Что делает
`testing`	банк шаблонов заданий, сборка попытки, адаптивный выбор, оценка θ по Рашу, грейд, кулдаун
`profile`	внешние учётные записи (FSP ID) и подтверждённые результаты из источников
`matching`	скор кандидата для потребности, объяснение, ранжирование, базовая линия

Ядро `testing` и `matching` — чистые функции без БД, поэтому одни и те же функции вызываются из API и из скриптов валидации.

Роли и права

Роль	Может
Кандидат	профиль, согласие на обработку ПДн, привязка FSP ID, выбор специализации и грейда, тест, приглашения, отклики
Работодатель	профиль компании, потребности и вакансии, категории и подборка, карточки кандидатов без контактов, приглашения

Контакты кандидата работодатель видит только при принятом приглашении или собственном отклике кандидата. Чужие данные по прямому адресу недоступны (403).

2. Механика тестирования и устойчивость к утечке заданий

Категория и грейд

- Категория = специализация (`backend`, `frontend`, `bas` — БАС-разработчик) + грейд (`junior`, `middle`, `senior`).
- Кандидат **сам выбирает** грейд, тест его подтверждает. Пороги уровня θ: junior ≥ −0,5; middle ≥ 0,5; senior ≥ 1,5.
- Статус: `confirmed` (подтверждён), `unconfirmed` (не подтверждён), `none` (тест не проходил). Не подтвердивший грейд кандидат не скрывается: он виден в подборке ниже подтверждённых (множитель 0,6 к скору). Принудительного понижения нет.
- Если θ выше порога следующего грейда, кандидату предлагается тест на повышение.

Кулдаун

Повторный тест на тот же или более высокий грейд — не раньше чем через 30 дней: за это время можно реально подтянуть навыки, а перебор попыток «до везения» теряет смысл. Тест на грейд ниже сразу после провала — без ожидания и без сжигания попытки. На демо срок сокращён флагом `DEMO_COOLDOWN_MIN`.

Шаблоны заданий

Шаблон = функция `generate(seed)` и метаданные `{id, spec, topic, b, kind}`. Сид = hash(candidate_id, attempt_id, template_id): у каждого кандидата свои числа, имена, фрагменты кода и порядок вариантов, правильный ответ вычисляет код. Виды заданий: выбор одного варианта, числовой ответ («что выведет код»), упорядочивание шагов.

Оценка по модели Раша и адаптивность

Вероятность верного ответа кандидата на задание:

$$P = 1 / (1 + \exp(-(\theta - b)))$$

где P — вероятность верного ответа (от 0 до 1); θ — уровень кандидата (логиты, безразмерная шкала); b — сложность шаблона (логиты, та же шкала).

Экспресс-тест — 10 заданий за 5 минут, темы покрывают специализацию. Первые три задания — около порога заявленного грейда, дальше адаптивно: верный ответ — следующий шаблон сложнее, неверный — проще. θ оценивается методом максимального правдоподобия при известных b, с ограничением [−3; 3]. Начальные b заданы экспертно и уточняются на симуляции (раздел 4).

Почему задания не утекают

1. **Значения уникальны для кандидата.** Слитый ответ относится к чужому варианту и не подходит к своему.

2. **Пул шаблонов на тему**, случайный выбор среди шаблонов близкой сложности — разные кандидаты получают разные наборы.
3. **Лимит времени** на задание и на тест мешает искать ответ на стороне.
4. **Статистика шаблона:** если доля верных ответов по шаблону резко растёт, шаблон помечается «скомпрометирован» и выводится из пула (концепция).

Количественная проверка — раздел 4, подраздел А4: сценарий «кандидат знает ответы всех вариантов, выданных 50 другим кандидатам» сравнивается с гипотетическим единым тестом для всех.

3. Механика подбора и категоризации

Обратный сценарий: работодатель находит кандидата

1. Работодатель создаёт потребность: специализация, грейд, навыки с весами 1–3, описание, **зарплата от–до в рублях с пометкой «гросс» или «нетто»** (без неё сохранить нельзя), формат работы.
2. Система показывает категории с числом кандидатов.
3. Ранжированная подборка с объяснением каждого места. Уточнение фильтров не сбрасывает подборку: исходный запрос хранится, фильтры — параметры запроса.
4. Адресное приглашение: сообщение, зарплата от–до, гросс или нетто, способ связи.
5. Кандидат принимает приглашение — работодателю открываются контакты. Фамилию, имя и места работы видно сразу, телефон, email и Telegram — только после согласия.

Жёсткий фильтр

Специализация и грейд из категории; флажок «соседние грейды» расширяет фильтр на ступень вверх и вниз.

Скор кандидата

score = 100 · (0,40 · stack_match + 0,30 · test_score + 0,15 · fsp_score + 0,10 · freshness + 0,05 · pref_match) · k

где: - score — итоговый скор кандидата для потребности, баллы от 0 до 100; - stack_match — доля веса требуемых навыков, которые есть в профиле (вес навыка задаёт работодатель), от 0 до 1; - test_score — θ из теста, нормированная в [0; 1] внутри категории; - fsp_score — вклад подтверждённых результатов ФСП, от 0 до 1: призовые места и число стартов; дисциплина, близкая к специализации, — множитель 1,5; при отсутствии истории ФСП — 0; - freshness — давность теста и регулярных заданий, от 0 до 1; - pref_match — совпадение формата работы, города и зарплатных ожиданий с вилкой, от 0 до 1; - k — 1 для подтверждённого грейда и 0,6 для неподтверждённого (безразмерный).

Результаты ФСП влияют только на порядок **внутри** категории и не заменяют тест: все дисциплины равны, буст только за близость дисциплины к специализации (backend и frontend — «продуктовое программирование»; БАС-разработчик — «программирование беспилотных авиационных систем» и «программирование робототехники»).

Объяснение

Каждый ответ подбора содержит список факторов `{factor, weight, value, text}`, например: «Подтвердил middle (θ = 0,9, 82-й процентиль категории)», «Призёр ФСП по продуктовому программированию, 2026», «Нет истории ФСП — оценка по тесту и навыкам».

Кандидат без истории ФСП

Путь не блокируется ни на одном шаге: регистрация по email, профиль, тест, категория. Компонент ФСП у такого кандидата равен нулю, но он весит 15 %, а тест и навыки — 70 %, поэтому сильный

кандидат без ФСП стоит выше слабого с ФСП (проверка — в разделе 4).

4. Процедура валидации и результаты

Процедура

1. Синтетическая база (`data/gen.py`, seed = 42): у каждого кандидата есть скрытая способность θ_true, которую не видят ни тест, ни подбор.
2. **Тестирование:** каждый кандидат проходит симуляцию теста дважды (ответ верен с вероятностью по модели Раша от θ_true); считаются повторяемость, точность против скрытой правды, разделяющая сила шаблонов, устойчивость к утечке.
3. **Подбор:** эталон пар «потребность — кандидат» размечен по скрытой правде, а не по формуле подбора (правило ниже), поэтому проверка не замкнута сама на себя. Сравнение с поиском по ключевым словам.
4. **Вторая независимая разметка** 200 пар (10 потребностей × 20 карточек) сделана вслепую: разметчик видел только карточки (специализация, заявленный грейд, навыки, стаж, результаты ФСП) без θ_true, без скора системы и без первой разметки. Разметку выполнила языковая модель в отдельном контексте; сомнительные случаи — `validation/stasya_doubts.md`. Согласие разметчиков — каппа Коэна.

Правило эталонной разметки

Labels are computed from the hidden truth, **not** from the matching formula, so the check is not circular.

- `grade_by_theta_true`: junior >= -0.5, middle >= 0.5, senior >= 1.5 (CONTRACT section 2); below -0.5 means "below junior".
- `coverage` = sum of need skill weights present in the candidate's skills / sum of all need skill weights.
- **2**: same spec, `grade_by_theta_true` == need grade, coverage >= 0.7.
- **1**: same spec, grade equal or adjacent (one step; "below junior" is adjacent to junior), coverage >= 0.4 (and not 2).
- **0**: everything else (not written to the file).

Why ranking by resume alone is not enough

- Experience and number of skills correlate with theta, but with noise (corr(theta, experience) = 0.69).
- `claimed_grade`: 70 % honest by theta, 20 % overstated by one step, 10 % understated.

Distributions (seed 42)

- spec: backend 135, frontend 135, bas 30; minors 16-17 y.o.: 15; with FSP history: 120 (40 %).
- grade by theta_true: below junior 98, junior 100, middle 80, senior 22. Claimed: junior 164, middle 105, senior 31.
- Pairs labeled 2 per need: from 0 to 23 (one need has no "2" — kept as is, reported honestly); labeled 1: 8 to 80.

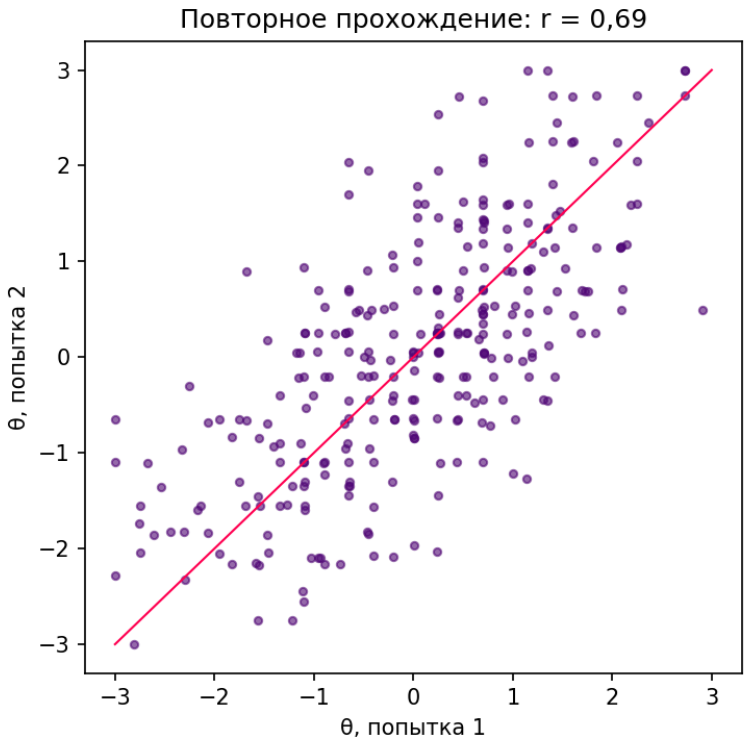
> Цифры получены на заглушке ядра (validation/_stub.py), не на коде бэкенда. Перед сдачей — перепрогнать.

Ядро: **STUB (validation/_stub.py) — backend core not importable: ModuleNotFoundError: No module named 'app'** Сгенерировано: `python -X utf8 validation/run_all.py`, данные seed = 42, 300 кандидатов, 30 потребностей.

A. Тестирование

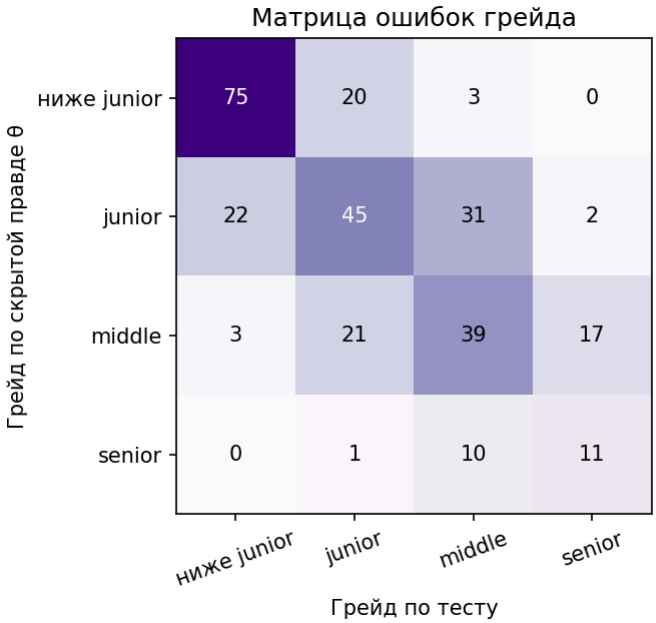
A1. Повторное прохождение (test-retest)

- Совпадение грейда по тесту между двумя попытками (разные варианты заданий): **49,7 %**
- Совпадение решения «подтверждён / не подтверждён»: **71,7 %**
- Корреляция Пирсона θ_1 и θ_2: **0,69**



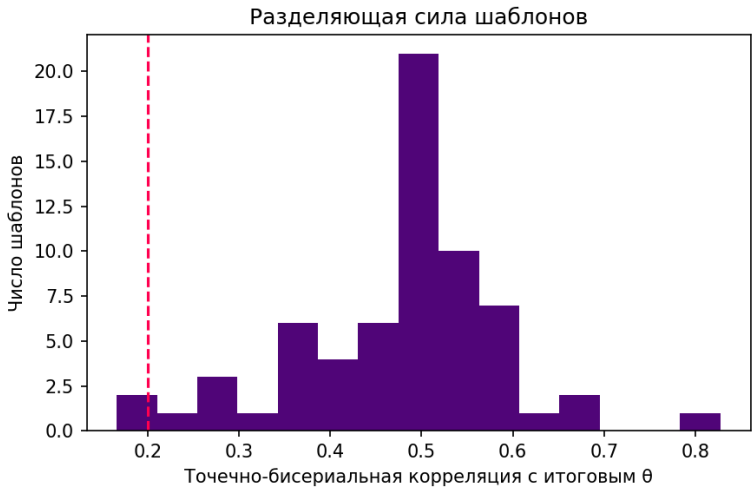
A2. Точность против скрытой правды

- Точное совпадение грейда: **56,7 %**; ошибка больше чем на ступень: **3,0 %**
- Решение о подтверждении заявленного грейда верно: **76,3 %**
- Средняя абсолютная ошибка θ: **0,58**



A3. Разделяющая сила шаблонов

- Шаблонов с ≥ 10 ответами: 65; медиана точечно-бисериальной корреляции: **0,49**
- Корреляция заявленной и уточнённой сложности b: **0,98**
- На доработку (корреляция < 0,2): **2**



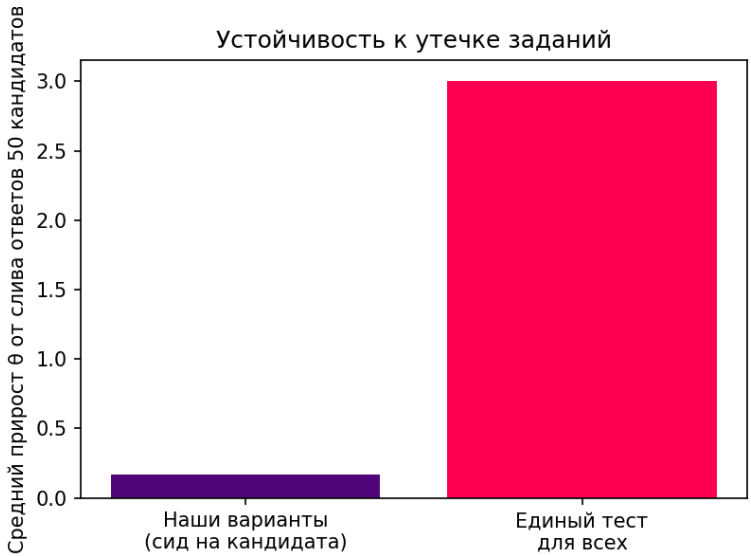
Шаблон	Ответов	r_pb	Доля верных, нижняя треть	Верхняя треть	b заявл.	b уточн.
backend.t3.5	16	0,17	0,20	0,40	2,65	3,12
bas.t3.2	43	0,18	0,50	0,71	-0,05	-0,22
bas.t3.4	14	0,23	0,25	0,50	1,75	2,13
bas.t3.3	33	0,25	0,36	0,73	0,85	0,67
frontend.t2.0	57	0,27	0,63	0,84	-1,95	-2,19
backend.t0.4	102	0,28	0,24	0,56	1,45	1,57
frontend.t3.5	11	0,33	0,00	0,33	2,65	2,45
frontend.t3.2	218	0,36	0,29	0,68	-0,05	-0,08
backend.t1.4	99	0,36	0,12	0,55	1,55	1,87

Шаблон	Ответов	r_pb	Доля верных, нижняя треть	Верхняя треть	b заявл.	b уточн.
frontend.t2.4	73	0,36	0,08	0,54	1,65	1,79
backend.t2.4	63	0,36	0,24	0,62	1,65	1,73
frontend.t1.0	31	0,37	0,50	0,90	-2,05	-2,79
bas.t3.1	36	0,38	0,33	0,83	-0,95	-0,91
frontend.t1.5	29	0,40	0,11	0,44	2,45	3,08
frontend.t0.3	194	0,41	0,23	0,69	0,55	0,55

Показаны 15 шаблонов с наименьшей корреляцией; полный список — `results.json`.

A4. Устойчивость к утечке

- Доля пар кандидатов с одинаковым вариантом на одном шаблоне: **0,2 %**
- Слив ответов 50 других кандидатов: средний прирост θ **0,166**, максимум 1,32, прирост хоть какой-то у 29,0 % из 100 «сливщиков»
- Для сравнения, единый тест для всех: средний прирост θ **3,00**



A5. Согласие разметчиков

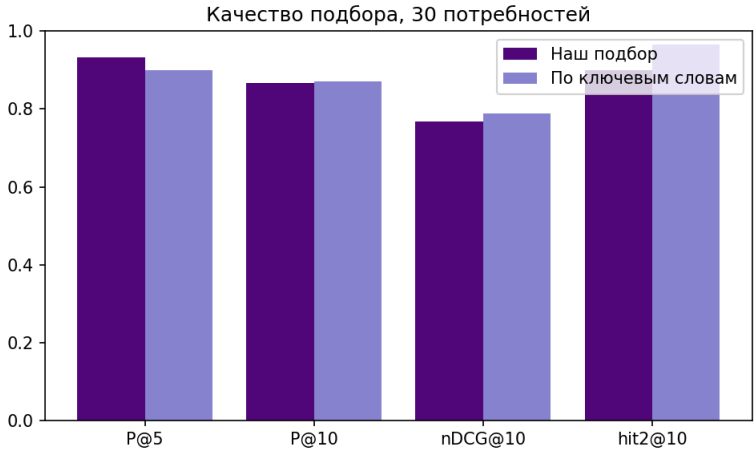
- Пар: 200; совпадение: **77,5 %**; каппа Коэна: **0,64**; квадратично-взвешенная каппа: **0,78**

На 10 потребностях × 20 карточек	P@5	P@10	nDCG@10
Наш подбор (соседние грейды) против независимой разметки	0,92	0,83	0,84
Ключевые слова против независимой разметки	0,94	0,91	0,94
Наш подбор против разметки по скрытой правде	0,92	0,84	0,74

B. Подбор

Метод	P@5	P@10	nDCG@10	Есть «2» в топ-10
Наш подбор (грейд строго)	0,93	0,87	0,77	90,0 %
Наш подбор (соседние грейды)	0,99	0,95	0,71	96,7 %

Метод	P@5	P@10	nDCG@10	Есть «2» в топ-10
По ключевым словам	0,90	0,87	0,79	96,7 %



Кандидаты без истории ФСП

- Доля без ФСП в топ-10: **46,6 %** при доле в базе 60,1 %
- Сильный кандидат без ФСП (разметка 2) выше слабого с ФСП (разметка 0): **91,3 %** из 3173 сравнений
- Пример: потребность 1 — кандидат 54 без ФСП на 25-м месте, кандидат 57 с ФСП (не подходит) на 27-м.

C. Производительность

- `rank` на 300 кандидатах: **0,2 мс**; на 3000: **1,5 мс** (медиана из 20 прогонов)

5. Интеграция с реестром ФСП и FSP ID (Keycloak)

Поток входа (OIDC, authorization code)

1. Кандидат нажимает «Войти через FSP ID»; бэкенд перенаправляет на `/auth/fsp/authorize` с `client_id`, `redirect_uri`, `state` и `scope=openid profile email`.
2. Keycloak ФСП аутентифицирует пользователя и возвращает код на `/auth/fsp/callback`.
3. Бэкенд обменивает код на `id_token` и `access_token`, проверяет подпись и `state`, читает claims.
4. Создаётся или находится внешняя учётная запись `ExternalIdentity(provider='fsp', subject=sub)`, уникальная по паре (provider, subject), и привязывается к пользователю.
5. По `subject` запрашиваются результаты участника из реестра ФСП и импортируются как подтверждённые результаты.

На демо Keycloak имитируется: те же эндпоинты и тот же обмен, данные берутся из `data/fsp_registry.json` (все кандидаты с FSP ID и 5 «свободных» участников для показа привязки).

Какие поля берём и зачем

Поле	Откуда	Зачем
`sub`	id_token	ключ внешней учётной записи
`name`, `email`	id_token	предзаполнение профиля (кандидат может исправить)
соревнование, дисциплина, год, место, роль, командный или личный зачёт	реестр ФСП	подтверждённый результат, компонент `fsp_score` и объяснение в подборке

Подтверждённый результат и источник

`VerifiedResult(source, external_id, event, discipline, year, place, role, team)` — подтверждённый результат из источника результатов. Поле `source` сейчас равно `fsp`; другие источники (олимпиады, хакатоны) добавляются без изменения схемы. Термин — «подтверждённый результат», не «достижение» (см. `CONTEXT.md`).

Замена имитации на боевой Keycloak

1. Зарегистрировать клиента в realm ФСП (confidential client, redirect URI стенда).
2. Указать в `.env` адрес realm, `client_id`, `client_secret`; адреса authorize, token и jwks берутся из `/.well-known/openid-configuration`.
3. Заменить источник результатов с файла на API реестра ФСП — функция `profile.import_results(source, claims, registry)` принимает данные и не зависит от транспорта.

Если истории ФСП нет

Кандидат выбирает «У меня нет истории ФСП» или привязывает FSP ID без результатов. Путь не блокируется, `fsp_score = 0`, в объяснении подборки — «нет истории ФСП», оценка идёт по тесту и навыкам.

6. Описание API

Префикс `/api`, авторизация — JWT в заголовке `Authorization: Bearer`. Полная спецификация — Swagger стенда `/docs` и файл `docs/openapi.json`.

> OpenAPI бэкенда недоступен (ModuleNotFoundError); ниже — таблица из CONTRACT.md, раздел 6.

Метод	Путь	Кто	Суть
POST	/auth/register	все	email, password, role → код подтверждения (на демо возвращается в ответе и пишется в лог)
POST	/auth/verify	все	email, code
POST	/auth/login	все	→ token
GET	/auth/fsp/authorize, /auth/fsp/callback	кандидат	имитация OIDC FSP ID: создаёт ExternalIdentity и импортирует VerifiedResult
POST	/auth/demo/{role}	все	мгновенный вход демо-аккаунтом для жюри
GET/PUT	/me/profile	кандидат	профиль, согласие, приватность
POST	/me/resume/parse	кандидат	загрузка PDF → черновик полей (pdfplumber + регулярные выражения, LLM опционально)
GET	/me/profile.pdf	кандидат	стандартизированный PDF-профиль
GET	/me/results	кандидат	подтверждённые результаты (по источникам)
GET	/me/category	кандидат	текущая категория, история попыток, `next_attempt_at`
POST	/tests/start	кандидат	{spec, claimed_grade} → attempt_id, первое задание (403 при кулдауне с датой)
POST	/tests/{id}/answer	кандидат	{item_id, answer} → следующее задание или итог
GET	/tests/{id}/result	кандидат	θ, грейд, разбор
GET	/me/invitations; POST /invitations/{id}/accept, /decline	кандидат	
GET	/vacancies; POST /vacancies/{id}/apply; GET /me/applications	кандидат	дополнительно
GET/PUT	/employer	работодатель	профиль компании
POST	/needs	работодатель	создать потребность → search_id
GET	/needs/{id}/categories	работодатель	категории с количеством кандидатов

Метод	Путь	Кто	Суть
GET	/needs/{id}/matches?grade_adj=&fsp_only=&skills=&limit=	работодатель	ранжированная подборка с `explain`, контакты скрыты
GET	/candidates?spec=&grade=&skills=&fsp=&status=	работодатель	поиск по банку
GET	/candidates/{id}	работодатель	карточка; контакты только при accepted-приглашении или отклике
POST	/invitations	работодатель	{candidate_id, need_id?, message, salary_from, salary_to, salary_type, contact_method}
GET	/employer/invitations	работодатель	статусы
POST	/needs/{id}/publish; GET /needs/{id}/applications	работодатель	дополнительно

7. Библиотеки и компоненты с версиями

> Нет backend/requirements.txt.
>
> Нет frontend/package.json.
>
> Нет docker-compose.yml.

Валидация и документация

- Python 3.12+, matplotlib, numpy (графики валидации); markdown (сборка документации).

8. Сборка, развёртывание и локальный запуск

```
git clone https://github.com/Wit-Sky/fsp-lct2026
cd fsp-lct2026
cp .env.example .env
docker compose up --build
```

После старта: веб-приложение — http://localhost/, Swagger — http://localhost/api/docs. Демо-доступы: кнопки «Войти как кандидат» и «Войти как работодатель» на главной (`candidate@demo.fsp`, `employer@demo.fsp`).

> Раздел будет уточнён по README и docker-compose.yml капитана (docs/deploy.md).

9. Концепции за рамками MVP

Защита от фиктивных вакансий

- Рейтинг ответов работодателя: доля приглашений, по которым работодатель ответил кандидату в срок.
- Доля приглашений без найма: аномально высокая — сигнал «сбор контактов», ограничение числа приглашений.
- Подтверждение компании по ИНН как добровольная метка доверия.
- Зарплата от–до обязательна — вакансию без денег опубликовать нельзя.

Интеграция с ATS

- Вебхуки статусов приглашений и откликов (sent, viewed, accepted, declined).
- Экспорт карточки кандидата (с его согласия) в JSON и HR-XML.

Регулярные задания от работодателей

Работодатель публикует задание по специализации со свободным ответом; ответы освежают профиль кандидата (компонент freshness) и дают работодателю повод для приглашения.

Кандидаты 16–17 лет и 152-ФЗ

- Дата рождения в профиле; младше 18 лет — обязательное согласие родителя, работодатель видит пометку «несовершеннолетний».
- Согласие на обработку персональных данных обязательно; кандидат управляет видимостью полей; контакты раскрываются только с его согласия.